Autochip Report

Yanbo Wang, yw7636

October 11, 2026

1 Examples

1.1 Example 1: binary_to_bcd

```
//I am trying to create a Verilog model binary_to_bcd_converter for a binary to binary-coded-decimal converter. It must meet the following specifications:
//  - Inputs:
//      - Binary input (5-bits)
//  - Outputs:
//      - BCD (8-bits: 4-bits for the 10's place and 4-bits for the 1's place)
//
//How would I write a design that meets these specifications?

module binary_to_bcd_converter (
    input [4:0] binary_input,
    output [7:0] bcd_output
);
```

Figure 1: Natural-language function description and module template

For this example, the natural-language function description I used in the prompt file is the same as the given default prompt in the previous ChipChat lab. The module template uses the exact module name and ports expected by the testbench.

The testbench used is the provided testbench from GitHub. A mismatch counter was added to count and display the number of failed checks for a module. The output format was adjusted to make the testbench compatible with the AutoChip code.

The parameter setup is included in the config.json file:



Figure 2: AutoChip configuration for binary_to_bcd

The model I used for this example is Claude Sonnet 4.5, with an iteration limit of 5 and 5 candidates per iteration.

Here is the printed result from the code cell in the notebook:

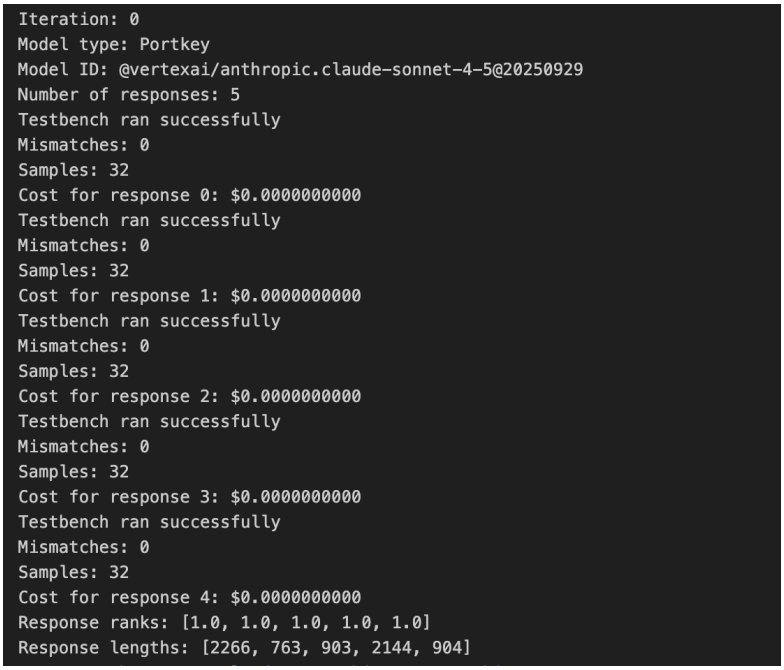


Figure 3: AutoChip generation and test results

All 5 candidates in the first iteration passed all 32 testbench cases. The best response was response number 1, with the shortest generated code. Since the candidates passed at iteration 0, no feedback-driven corrections were needed.

```
        // For 5-bit input, maximum value is 31
        if (binary_input < 5'd10) begin
            tens = 4'd0;
            ones = binary_input[3:0];
        end
        else if (binary_input < 5'd20) begin
            tens = 4'd1;
            ones = binary_input - 5'd10;
        end
        else if (binary_input < 5'd30) begin
            tens = 4'd2;
            ones = binary_input - 5'd20;
        end
        else begin // binary_input >= 30 and <= 31
            tens = 4'd3;
            ones = binary_input - 5'd30;
        end
    end
```

Figure 4: Core part of the module code

For response 1, the code compares the input against decimal ranges. For each range, it assigns a fixed tens digit and subtracts the corresponding multiple of ten to obtain the ones digit.

Here are the iverilog and vvp commands used, including the simulation output:

```
yanbowang@YanbodeMacBook-Air binary_to_bcd_example % iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb \
  -o outputs_binary_to_bcd/iter0/response1/binary_to_bcd_converter.vvp \
  outputs_binary_to_bcd/iter0/response1/binary_to_bcd_converter.sv \
  binary_to_bcd_tb.v
yanbowang@YanbodeMacBook-Air binary_to_bcd_example % vvp -n outputs_binary_to_bcd/iter0/response1/binary_to_bcd_converter.vvp
Testing Binary-to-BCD Converter...
VCD info: dumpfile my_design.vcd opened for output.
Mismatches: 0 in 32 samples
yanbowang@YanbodeMacBook-Air binary_to_bcd_example %
```

Figure 5: Compilation and simulation commands and output

1.2 Example 2: sequence_detector

```
//I am trying to create a Verilog model for a sequence detector. It must meet the following specifications:
//  - Inputs:
//      - Clock
//      - Active-low reset
//      - Data (3 bits)
//  - Outputs:
//      - Sequence found
//
//While enabled, it should detect the following sequence of binary input values:
//  - 0b001
//  - 0b101
//  - 0b110
//  - 0b000
//  - 0b110
//  - 0b110
//  - 0b011
//  - 0b101
//
//How would I write a design that meets these specifications?


module sequence_detector (
    input clk,
    input reset_n,
    input [2:0] data,
    output sequence_found
);
```

Figure 6: Natural-language function description and module template

For this example, the natural-language function description I used in the prompt file is the same as the given default prompt in the previous ChipChat lab. The module template uses the exact module name and ports expected by the testbench.

The testbench used is the provided testbench from GitHub. Mismatch and sample counters were added to count and display the number of failed checks and total checks. The output format was adjusted to make the testbench compatible with the AutoChip code.

The parameter setup is included in the config.json file:

```json
{
    "general": {
        "prompt": "./sequence_detector.sv",
        "name": "sequence_detector",
        "testbench": "./sequence_detector_tb.v",
        "model_family": "Portkey",
        "model_id": "@vertexai/anthropic.claude-sonnet-4-5@20250929",
        "num_candidates": 5,
        "iterations": 5,
        "outdir": "outputs_sequence_detector",
        "log": "log.txt",
        "ensemble": false
    }
}
```


Figure 7: AutoChip configuration for sequence_detector

The model I used for this example is Claude Sonnet 4.5, with an iteration limit of 5 and 5 candidates per iteration.

Here is the printed result from the code cell in the notebook:

```
Iteration: 0
Model type: Portkey
Model ID: @vertexai/anthropic.claude-sonnet-4-5@20250929
Number of responses: 5
Simulation error
Mismatches: 2
Samples: 12
Cost for response 0: $0.0000000000
Simulation error
Mismatches: 2
Samples: 12
Cost for response 1: $0.0000000000
Simulation error
Mismatches: 2
Samples: 12
Cost for response 2: $0.0000000000
Simulation error
Mismatches: 2
Samples: 12
Cost for response 3: $0.0000000000
Simulation error
Mismatches: 2
Samples: 12
Cost for response 4: $0.0000000000
Response ranks: [0.8333333333333334, 0.8333333333333334, 0.8333333333333334, 0.8333333333333334, 0.8333333333333334]
Response lengths: [3411, 3553, 3541, 3581, 3370]
```

Figure 8: Iteration 0 results: all five candidates failed 2 of 12 checks

```
Iteration: 1
Model type: Portkey
Model ID: @vertexai/anthropic.claude-sonnet-4-5@20250929
Number of responses: 5
Simulation error
Mismatches: 2
Samples: 12
Cost for response 0: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 12
Cost for response 1: $0.0000000000
Simulation error
Mismatches: 2
Samples: 12
Cost for response 2: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 12
Cost for response 3: $0.0000000000
Simulation error
Mismatches: 2
Samples: 12
Cost for response 4: $0.0000000000
Response ranks: [0.8333333333333334, 1.0, 0.8333333333333334, 1.0, 0.8333333333333334]
Response lengths: [3708, 3444, 3708, 3429, 3709]
```

Figure 9: Iteration 1 results: responses 1 and 3 passed all 12 checks

In the first iteration, no candidate passed the testbench without errors. All five candidates had 2 mismatches across 12 checks. In the next iteration, two candidates successfully passed the testbench. The best response among these two candidates was response 3, which had a shorter length.

The log.txt file records the initial simulation errors and the feedback sent to the model. Here is the detailed feedback for the selected candidate from iteration 0, response 3:

```
user: The testbench simulated, but had errors. Please fix the module. The output of iverilog is as follows:
Error: Cycle 8, Expected: 1, Got: 0
Error: Cycle 9, Expected: 0, Got: 1
Mismatches: 2 in 12 samples
```

Figure 10: Simulation feedback showing mismatches at cycles 8 and 9

Original line:

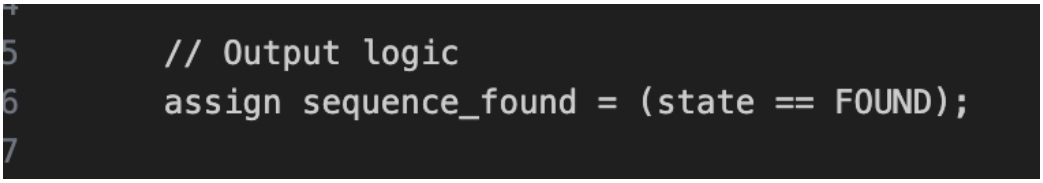


Figure 11: Original Moore-style output condition

Edited line:

```
// Output logic - assert when in S7 and receiving final value 101
assign sequence_found = (state == S7) && (data == 3'b101);
```

Figure 12: Revised Mealy-style output condition

In iteration 0, the module was generated as a Moore-style FSM with a separate FOUND state. The testbench checks the output immediately at the rising clock edge, before the nonblocking state update takes effect. This caused detection to appear one check late, producing mismatches at cycles 8 and 9. The revised code removed the FOUND state and asserted the output when the current state was S7 and the input was the final sequence value, 101. This changed the output from Moore-style to Mealy-style and matched the testbench's checking timing. The selected revised candidate passed all 12 checks.

Here are the iverilog and vvp commands used, including the simulation output:

```
yanbowang@YanbodeMacBook-Air sequence_detector_example % iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb \
  -o outputs_sequence_detector/iter1/response1/sequence_detector.vvp \
  outputs_sequence_detector/iter1/response1/sequence_detector.sv \
  sequence_detector_tb.v
yanbowang@YanbodeMacBook-Air sequence_detector_example % vvp -n outputs_sequence_detector/iter1/response3/sequence_detector.vvp
Mismatches: 0 in 12 samples
```

Figure 13: Icarus Verilog compilation and simulation output for iteration 1, response 3

1.3 Example 3: shift_register

```
//I am trying to create a Verilog model for a shift register. It must meet the following specifications:
//  - Inputs:
//      - Clock
//      - Active-low reset
//      - Data (1 bit)
//      - Shift enable
//  - Outputs:
//      - Data (8 bits)
//
//How would I write a design that meets these specifications?


module shift_register (
    input clk,
    input reset_n,
    input data_in,
    input shift_enable,
    output [7:0] data_out
);
```

Figure 14: Natural-language function description and module template

For this example, the natural-language function description I used in the prompt file is the same as the given default prompt in the previous ChipChat lab. The module template uses the exact module name and ports expected by the testbench.

Mismatch and sample counters were added to the testbench to count and display the number of failed checks and total checks. The output format was adjusted to make the testbench compatible with the AutoChip code.

The parameter setup is included in the config.json file:

```
{
    "general": {
        "prompt": "./shift_register.sv",
        "name": "shift_register",
        "testbench": "./shift_register_tb.v",
        "model_family": "Portkey",
        "model_id": "@vertexai/anthropic.claude-sonnet-4-5@20250929",
        "num_candidates": 5,
        "iterations": 5,
        "outdir": "outputs_shift_register",
        "log": "log.txt",
        "ensemble": false
    }
}
```


Figure 15: AutoChip configuration for shift_register

The model I used for this example is Claude Sonnet 4.5, with an iteration limit of 5 and 5 candidates per iteration.

Here is the printed result from the code cell in the notebook:

```
Iteration: 0
Model type: Portkey
Model ID: @vertexai/anthropic.claude-sonnet-4-5@20250929
Number of responses: 5
Simulation error
Mismatches: 6
Samples: 8
Cost for response 0: $0.0000000000
Simulation error
Mismatches: 6
Samples: 8
Cost for response 1: $0.0000000000
Simulation error
Mismatches: 6
Samples: 8
Cost for response 2: $0.0000000000
Simulation error
Mismatches: 6
Samples: 8
Cost for response 3: $0.0000000000
Simulation error
Mismatches: 6
Samples: 8
Cost for response 4: $0.0000000000
Response ranks: [0.25, 0.25, 0.25, 0.25, 0.25]
Response lengths: [426, 426, 442, 426, 426]
```

Figure 16: Generation and test results for iteration 0

```
Iteration: 1
Model type: Portkey
Model ID: @vertexai/anthropic.claude-sonnet-4-5@20250929
Number of responses: 5
Testbench ran successfully
Mismatches: 0
Samples: 8
Cost for response 0: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 8
Cost for response 1: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 8
Cost for response 2: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 8
Cost for response 3: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 8
Cost for response 4: $0.0000000000
Response ranks: [1.0, 1.0, 1.0, 1.0, 1.0]
Response lengths: [426, 426, 426, 426, 426]
```

Figure 17: Generation and test results for iteration 1

In the first iteration, no candidate passed the testbench. All candidates failed 6 out of 8 checks. In the second iteration, all candidates fixed the issue and passed all 8 checks. They also had the same response length. AutoChip selected the first candidate, response 0, as the best response.

Here is the simulation feedback for response 0 from the first iteration:

```
Error: Test case 1 failed. Expected: 10000000, Got: 00000001
Error: Test case 2 failed. Expected: 01000000, Got: 00000010
Error: Test case 3 failed. Expected: 10100000, Got: 00000101
Error: Test case 4 failed. Expected: 01010000, Got: 00001010
Error: Test case 5 failed. Expected: 00101000, Got: 00010100
Error: Test case 7 failed. Expected: 10000000, Got: 00000001
Mismatches: 6 in 8 samples
```

Figure 18: Simulation feedback showing six mismatches

Original code:

```
        end else if (shift_enable) begin
            shift_reg <= {shift_reg[6:0], data_in};
        end
```

Figure 19: Original left-shift implementation

Based on the output and the code, the testbench expected a right shift with data_in entering bit 7. However, the initial design shifted left and inserted data_in into bit 0, causing the mismatches. The initial prompt did not specify the shift direction, so the simulation feedback clarified the behavior expected by the testbench. AutoChip corrected the shift direction in the next iteration:

```
    end else if (shift_enable) begin
        shift_reg <= {data_in, shift_reg[7:1]};
    end
```

Figure 20: Revised right-shift implementation

Here are the iverilog and vvp commands used, including the simulation output:

```
yanbowang@YanbodeMacBook-Air shift_register_example % iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb \
  -o outputs_shift_register/iter1/response0/shift_register.vvp \
  outputs_shift_register/iter1/response0/shift_register.sv \
  shift_register_tb.v
yanbowang@YanbodeMacBook-Air shift_register_example % vvp -n outputs_shift_register/iter1/response0/shift_register.vvp
Mismatches: 0 in 8 samples
```

Figure 21: Compilation and simulation commands and output

1.4 Example 4: abro_state_machine

```
//I am trying to create a Verilog model for an ABRO state machine. It must meet the following specifications:
//   - Inputs:
//        - Clock
//      - Active-low reset
//        - A
//        - B
//    - Outputs:
//        - O
//        - State
//
//Other than the main output from ABRO machine, it should output the current state of the machine for use in verification.
//
//The states for this state machine should be one-hot encoded.
//
//How would I write a design that meets these specifications?

module abro_state_machine (
    input clk,
    input rst_n,
    input A,
    input B,
    output O,
    output [3:0] State
);
```

Figure 22: Natural-language function description and module template

For this example, the natural-language function description I used in the prompt file is the same as the given default prompt in the previous ChipChat lab. The module template uses the exact module name and ports expected by the testbench.

Mismatch and sample counters were added to the testbench to count and display the number of failed checks and total checks. The output format was adjusted to make the testbench compatible with the AutoChip code.

The parameter setup is included in the config.json file:

```
{
    "general": {
        "prompt": "./abro_state_machine.sv",
        "name": "abro_state_machine",
        "testbench": "./abro_state_machine_tb.v",
        "model_family": "Portkey",
        "model_id": "@vertexai/anthropic.claude-sonnet-4-5@20250929",
        "num_candidates": 5,
        "iterations": 5,
        "outdir": "outputs_abro_state_machine",
        "log": "log.txt",
        "ensemble": false
    }
}
```


Figure 23: AutoChip configuration

The model I used for this example is Claude Sonnet 4.5, with an iteration limit of 5 and 5 candidates per iteration. Here is the printed result from the code cell in the notebook. Through iterations 0–5, Claude did not generate a response that passed the testbench. The entire printed result is too long to include in the report, so the result for the last iteration is displayed instead:

```
Iteration: 5
Model type: Portkey
Model ID: @vertexai/anthropic.claude-sonnet-4-5@20250929
Number of responses: 5
Simulation error
Mismatches: 10
Samples: 16
Cost for response 0: $0.0000000000
Simulation error
Mismatches: 7
Samples: 16
Cost for response 1: $0.0000000000
Simulation error
Mismatches: 10
Samples: 16
Cost for response 2: $0.0000000000
Simulation error
Mismatches: 10
Samples: 16
Cost for response 3: $0.0000000000
Simulation error
Mismatches: 10
Samples: 16
Cost for response 4: $0.0000000000
Response ranks: [0.375, 0.5625, 0.375, 0.375, 0.375]
Response lengths: [1768, 1998, 1791, 1765, 1805]
```

Figure 24: Results for the last iteration

AutoChip selected iteration 5, response 1, as the best response, which had the fewest mismatches: 7 out of 16 checks.

In iteration 0, the FSM's state encoding and completion behavior disagreed with the testbench, producing 14 mismatches. Iteration 1 changed the completion encoding to 0100 and held that state until reset, reducing mismatches to 10. Iterations 2–4 tried returning to IDLE, changing input priority, and making transitions depend on A and B, but still produced 10 mismatches. Finally, iteration 5 gated the output with (A —— B), reducing mismatches to 7.

```
// One-hot state encoding
localparam IDLE  = 4'b0001;
localparam GOT_A = 4'b0010;
localparam GOT_B = 4'b0100;
localparam DONE  = 4'b1000;
```

Figure 25: States for iteration 0

```
        DONE: begin
            next_state = IDLE;
        end
```

Figure 26: State transition for iteration 0

```
// One-hot state encoding
localparam IDLE   = 4'b0001;
localparam GOT_A  = 4'b0010;
localparam GOT_AB = 4'b0100;  // DONE state - both A and B seen
localparam GOT_B  = 4'b1000;
```

Figure 27: States for iteration 1

```
GOT_AB: begin
    // Stay in this state until reset
    next_state = GOT_AB;
end
```

Figure 28: State transition for iteration 1

```
// Output logic - O is high when both A and B have been seen
assign O = (current_state == GOT_AB);
assign State = current_state;
```

Figure 29: Iteration 4 output logic

```
// Output logic - O is high only when in GOT_AB state AND at least one input is high
assign O = (current_state == GOT_AB) && (A || B);
assign State = current_state;
```

Figure 30: Iteration 5 output logic

The remaining problems include state and output disagreements and test-bench timing issues, which requires more iterations to solve.

Here are the iverilog and vvp commands used, including the simulation output:

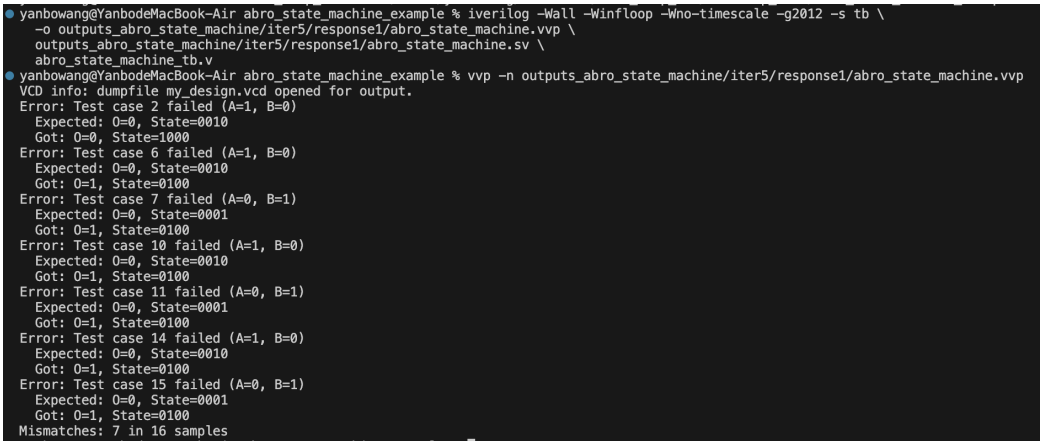


Figure 31: Compilation and simulation commands and output

1.5 Example 5: dice_roller

```
//
//I am trying to create a Verilog model for a simulated dice roller. It must meet the following specifications:
//    - Inputs:
//        - Clock
//        - Active-low reset
//        - Die select (2-bits)
//        - Roll
//    - Outputs:
//        - Rolled number (up to 8-bits)
//
//The design should simulate rolling either a 4-sided, 6-sided, 8-sided, or 20-sided die, based on the input die select. It should roll when the roll input goes
//
//How would I write a design that meets these specifications?



module dice_roller (
    input clk,
    input rst_n,
    input [1:0] die_select,
    input roll,
    output [7:0] rolled_number
);
```

Figure 32: Natural-language function description and module template

For this example, the natural-language function description I used in the prompt file is the same as the given default prompt in the previous ChipChat lab. The module template uses the exact module name and ports expected by the testbench.

Mismatch and sample counters were added to the testbench to count and display the number of failed checks and total checks. The output format was adjusted to make the testbench compatible with the AutoChip code.

The parameter setup is included in the config.json file:



```
{
    "general": {
        "prompt": "./dice_roller.sv",
        "name": "dice_roller",
        "testbench": "./dice_roller_tb.v",
        "model_family": "Portkey",
        "model_id": "@vertexai/anthropic.claude-sonnet-4-5@20250929",
        "num_candidates": 5,
        "iterations": 5,
        "outdir": "outputs_dice_roller",
        "log": "log.txt",
        "ensemble": false
    }
}
```

Figure 33: AutoChip configuration for dice_roller

The model I used for this example is Claude Sonnet 4.5, with an iteration limit of 5 and 5 candidates per iteration.

Here is the printed result from the code cell in the notebook:

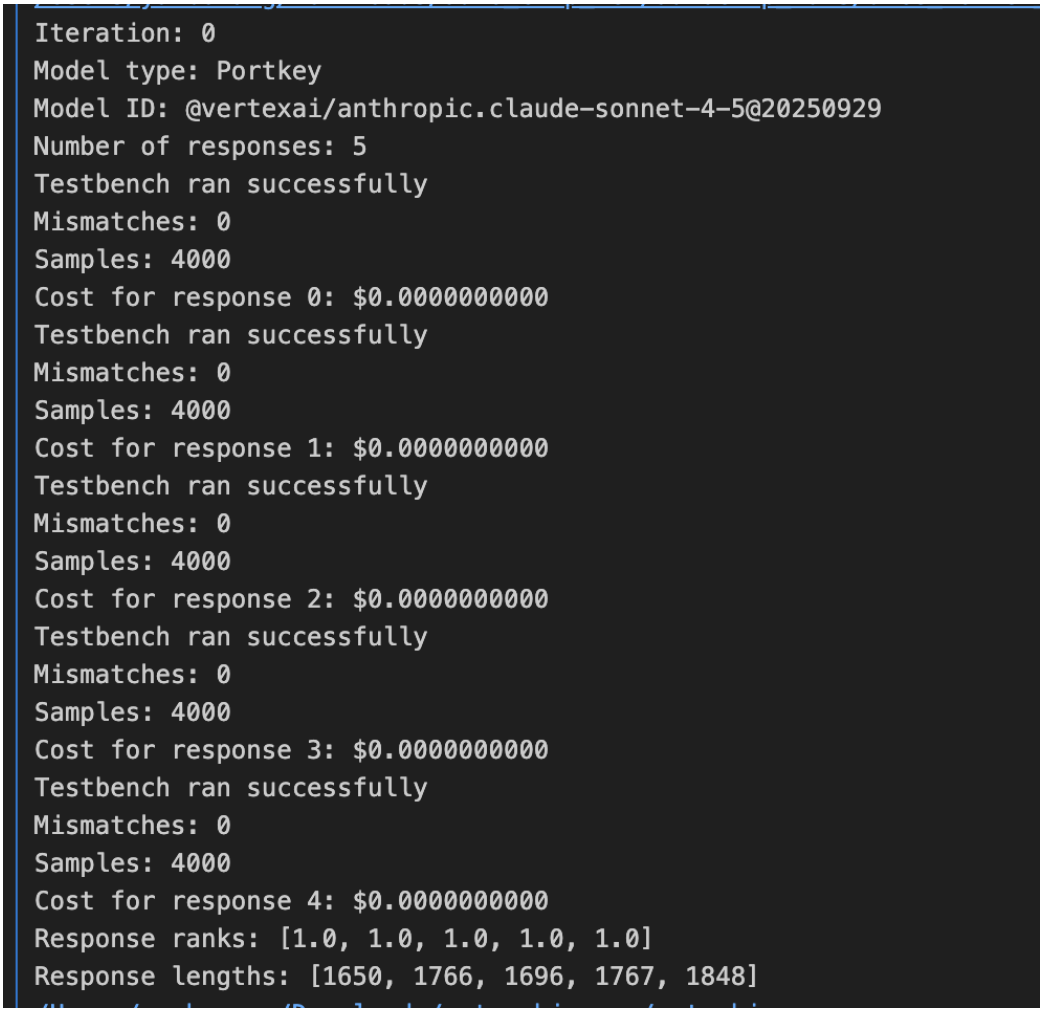


Figure 34: AutoChip generation and test results

All candidates pass the testbench on the first iteration. Autochip consider response 0 to be the best response as it has the least response length.

```
// Internal LFSR for pseudo-random number generation
reg [7:0] lfsr;

// Edge detection for roll signal
reg roll_prev;
wire roll_edge;

assign roll_edge = roll && !roll_prev;

// LFSR update and roll previous register
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        lfsr <= 8'b10101010; // Non-zero seed value
        roll_prev <= 1'b0;
    end else begin
        // Update LFSR every cycle for pseudo-randomness
        // Using polynomial x^8 + x^6 + x^5 + x^4 + 1
        lfsr <= {lfsr[6:0], lfsr[7] ^ lfsr[5] ^ lfsr[4] ^ lfsr[3]};

        // Store previous roll state for edge detection
        roll_prev <= roll;
    end
end
```

Figure 35: LFSR pseudorandom number generator

Response 0 uses an 8-bit LFSR to generate pseudorandom values, shifting its bits and inserting XOR feedback every clock cycle. When roll changes from low to high, it maps the current LFSR value into the selected die's range using modulo and adds one. The rolled number remains unchanged until the next roll.

Here are the iverilog and vvp commands used, including the simulation output:

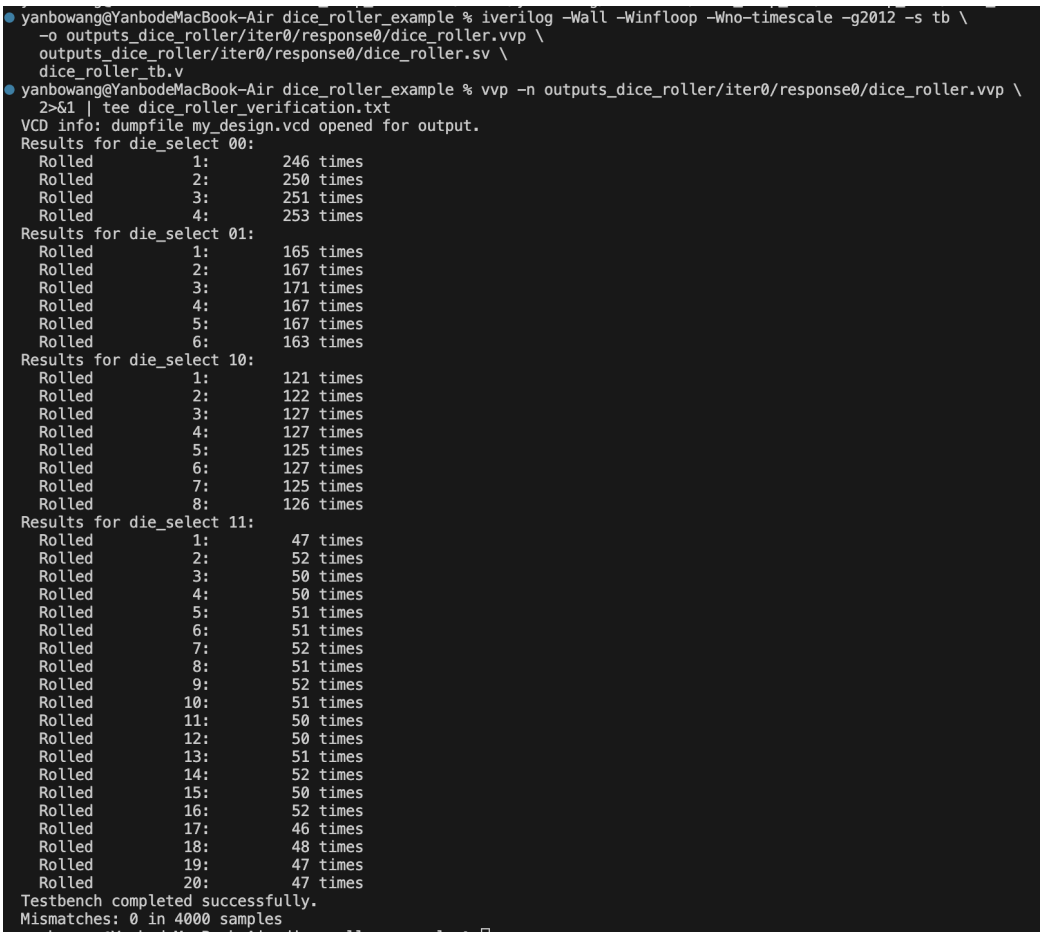


Figure 36: Compilation and simulation commands and output

1.6 Example 6: lfsr

```
//I am trying to create a Verilog model for an LFSR. It must meet the following specifications:
//  - Inputs:
//      - Clock
//        - Active-low reset
//  - Outputs:
//      - Data (8-bits)
//
//The initial state should be 10001010, and the taps should be at locations 1, 4, 6, and 7.
//
//How would I write a design that meets these specifications?


module lfsr (
    input clk,
    input reset_n,
    output [7:0] data
);
```

Figure 37: Natural-language function description and module template

For this example, the natural-language function description I used in the prompt file is the same as the given default prompt in the previous ChipChat lab. The module template uses the exact module name and ports expected by the testbench.

Mismatch and sample counters were added to the testbench to count and display the number of failed checks and total checks. The output format was adjusted to make the testbench compatible with the AutoChip code.

The parameter setup is included in the config.json file:



Figure 38: AutoChip configuration for the LFSR example

The model I used for this example is Claude Sonnet 4.5, with an iteration limit of 5 and 5 candidates per iteration. Here is the printed result from the code cell in the notebook. Through iterations 0–5, Claude did not generate a response that passed the testbench. The entire printed result is too long to include in the report, so the best result's iteration is displayed instead:

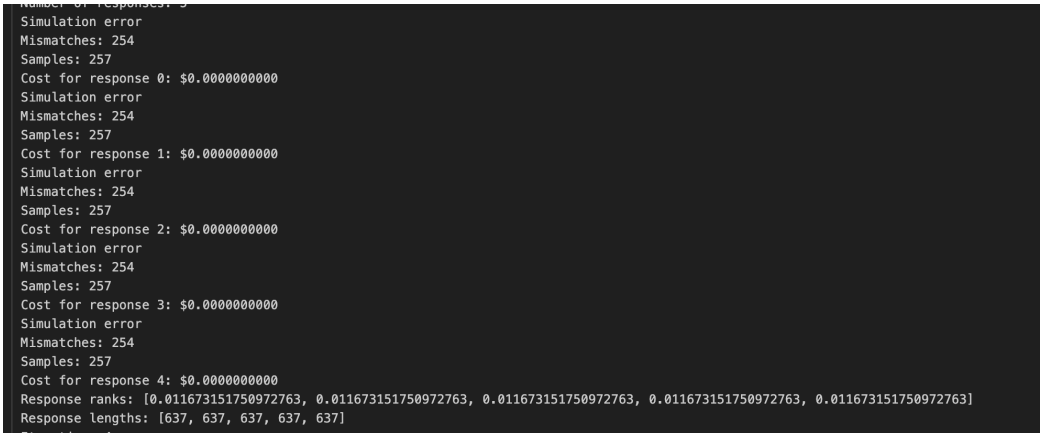


Figure 39: Iteration 3 results, including the selected best response

Autochip consider the best response to be iteration 0 with response number 3.

The prompt gives tap positions 1, 4, 6, and 7, but does not explain how to count them or which direction to shift. The generated code uses bit indices 1,

4, 6, and 7, while the testbench uses indices 0, 3, 5, and 6 and expects a left shift. AutoChip tries several changes over iterations 0–5, but does not find the correct combination. The best response still fails 254 out of 257 checks, passing only 3.

This issue remains for all 5 iterations.

Here are the iverilog and vvp commands used, including the simulation output:

```
yanbowang@YanbodeMacBook-Air lfsr_example % iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb \
  -o outputs_lfsr/iter0/response3/lfsr.vvp \
  outputs_lfsr/iter0/response3/lfsr.sv \
  lfsr_tb.v
yanbowang@YanbodeMacBook-Air lfsr_example % vvp -n outputs_lfsr/iter0/response3/lfsr.vvp
```

Figure 40: Icarus Verilog compilation and simulation output for the selected LFSR design

1.7 Example 7: sequence_generator

```
//
//I am trying to create a Verilog model for a sequence generator. It must meet the following specificatior
//  - Inputs:
//      - Clock
//      - Active-low reset
//      - Enable
//  - Outputs:
//      - Data (8 bits)
//
//While enabled, it should generate an output sequence of the following hexadecimal values and then repeat
//  - 0xAF
//  - 0xBC
//  - 0xE2
//  - 0x78
//  - 0xFF
//  - 0xE2
//  - 0x0B
//  - 0x8D
//
//How would I write a design that meets these specifications?


module sequence_generator (
    input clk,
    input reset_n,
    input enable,
    output [7:0] data
);
```

Figure 41: Natural-language function description and module template

For this example, the natural-language function description I used in the prompt file is the same as the given default prompt in the previous ChipChat lab. The module template uses the exact module name and ports expected by the testbench.

Mismatch and sample counters were added to the testbench to count and display the number of failed checks and total checks. The output format was adjusted to make the testbench compatible with the AutoChip code.

The parameter setup is included in the config.json file:

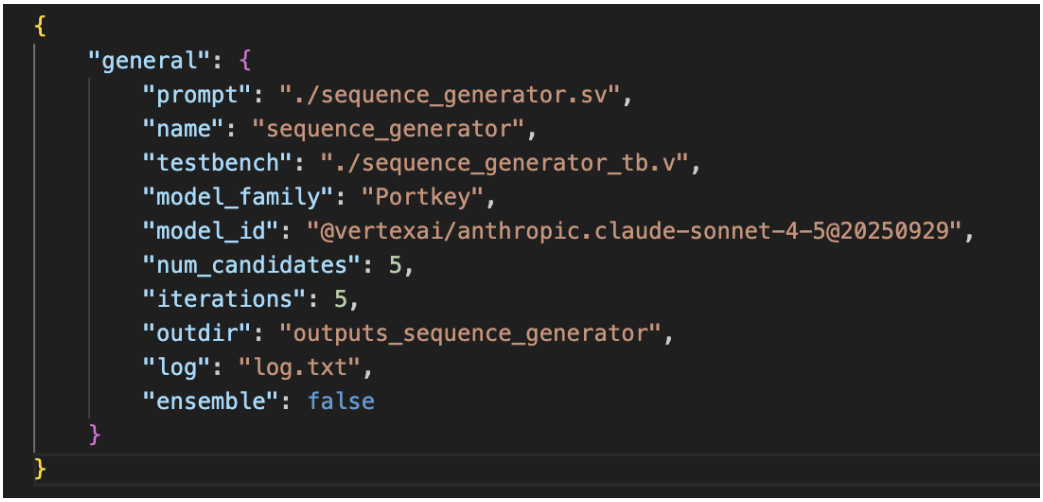


Figure 42: AutoChip configuration for sequence_generator

The model I used for this example is Claude Sonnet 4.5, with an iteration limit of 5 and 5 candidates per iteration. Here is the printed result from the code cell in the notebook.

```
Iteration: 0
Model type: Portkey
Model ID: @vertexai/anthropic.claude-sonnet-4-5@20250929
Number of responses: 5
Testbench ran successfully
Mismatches: 0
Samples: 17
Cost for response 0: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 17
Cost for response 1: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 17
Cost for response 2: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 17
Cost for response 3: $0.0000000000
Simulation error
Mismatches: 16
Samples: 17
Cost for response 4: $0.0000000000
Response ranks: [1.0, 1.0, 1.0, 1.0, 0.058823529411764705]
Response lengths: [1166, 1253, 1023, 1008, 1143]
```

Figure 43: Iteration 0 results: all five candidates passed the testbench

All five candidates passed the testbench with no mismatches in the first iteration. AutoChip selected response number 3 as the best response because it had the shortest response length.

```
// Output the appropriate value based on sequence index
case (seq_index + 1'b1)  // Use next index value
    3'd0: data <= 8'hAF;
    3'd1: data <= 8'hBC;
    3'd2: data <= 8'hE2;
    3'd3: data <= 8'h78;
    3'd4: data <= 8'hFF;
    3'd5: data <= 8'hE2;
    3'd6: data <= 8'h0B;
    3'd7: data <= 8'h8D;
endcase
```

Figure 44: Counter and case statement selecting the eight sequence values

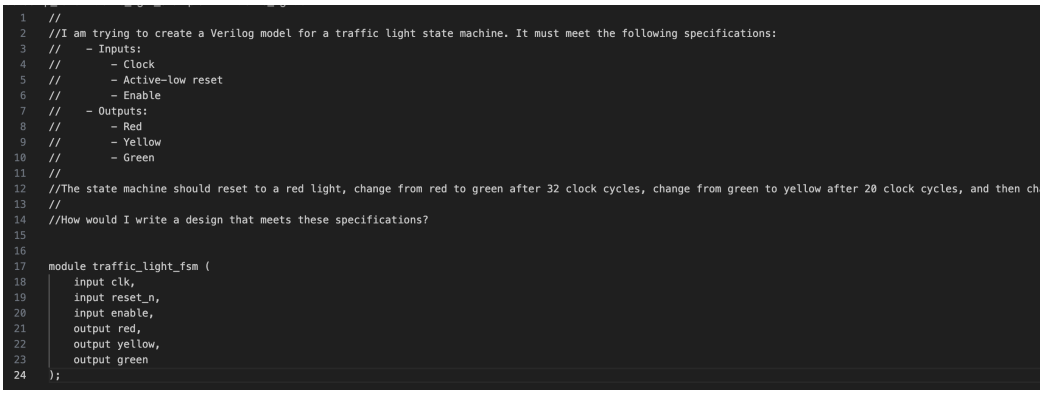
Response 3 uses a 3-bit counter to select the eight required sequence values and repeat the sequence. The counter and output update only when enabled. No further correction was needed.

Here are the iverilog and vvp commands used, including the simulation output:

```
yanbowang@YanbodeMacBook-Air sequence_generator_example % iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb \
  -o outputs_sequence_generator/iter0/response3/sequence_generator.vvp \
  outputs_sequence_generator/iter0/response3/sequence_generator.sv \
  sequence_generator_tb.v
yanbowang@YanbodeMacBook-Air sequence_generator_example % vvp -n outputs_sequence_generator/iter0/response3/sequence_generator.vvp
Test case passed: sequence generation
Test case passed: sequence repetition
Test case passed: disable sequence generator
Mismatches: 0 in 17 samples
```

Figure 45: Icarus Verilog compilation and simulation output for iteration 0, response 3

1.8 Example 8: traffic_light



```
//
//I am trying to create a Verilog model for a traffic light state machine. It must meet the following specifications:
//    - Inputs:
//        - Clock
//        - Active-low reset
//        - Enable
//    - Outputs:
//        - Red
//        - Yellow
//        - Green
//
//The state machine should reset to a red light, change from red to green after 32 clock cycles, change from green to yellow after 20 clock cycles, and then ch
//
//How would I write a design that meets these specifications?


module traffic_light_fsm (
    input clk,
    input reset_n,
    input enable,
    output red,
    output yellow,
    output green
);
```

Figure 46: Natural-language function description and module template

For this example, the natural-language function description I used in the prompt file is the same as the given default prompt in the previous ChipChat lab. The module template uses the exact module name and ports expected by the testbench.

Mismatch and sample counters were added to the testbench to count and display the number of failed checks and total checks. The output format was adjusted to make the testbench compatible with the AutoChip code.

The parameter setup is included in the config.json file:



Figure 47: AutoChip configuration for traffic_light

The model I used for this example is Claude Sonnet 4.5, with an iteration limit of 5 and 5 candidates per iteration. Here is the printed result from the code cell in the notebook.

```
Iteration: 0
Model type: Portkey
Model ID: @vertexai/anthropic.claude-sonnet-4-5@20250929
Number of responses: 5
Simulation error
Mismatches: 2
Samples: 4
Cost for response 0: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 4
Cost for response 1: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 4
Cost for response 2: $0.0000000000
Simulation error
Mismatches: 2
Samples: 4
Cost for response 3: $0.0000000000
Simulation error
Mismatches: 2
Samples: 4
Cost for response 4: $0.0000000000
Response ranks: [0.5, 1.0, 1.0, 0.5, 0.5]
Response lengths: [1545, 1807, 1901, 2937, 2418]
```

Figure 48: Iteration 0 results: responses 1 and 2 passed all four checks

Responses 1 and 2 passed the testbench with no mismatches in the first iteration. AutoChip selected response number 1 as the best response because it had the shorter response length among the passing candidates. No further correction was needed.

```
// State transition logic (combinational)
always @(*) begin
    case (current_state)
        RED: begin
            if (counter == 6'd31)
                next_state = GREEN;
            else
                next_state = RED;
        end

        GREEN: begin
            if (counter == 6'd19)
                next_state = YELLOW;
            else
                next_state = GREEN;
        end

        YELLOW: begin
            if (counter == 6'd6)
                next_state = RED;
            else
                next_state = YELLOW;
        end

        default: next_state = RED;
    endcase
end
```

Figure 49: State transition logic and counter thresholds

The code uses three states—RED, GREEN, and YELLOW—and a counter to switch lights after 32, 20, and 7 enabled clock cycles. The counter resets at each transition. An active-low reset returns the FSM to RED, while disabling `enable` pauses both the state and counter.

Here are the iverilog and vvp commands used, including the simulation output:

```
yanbowang@YanbodeMacBook-Air traffic_light_example % iverilog -Wall -Winfloop -Wno-timescale -g2012 -s tb \
  -o outputs_traffic_light/iter0/response1/traffic_light_fsm.vvp \
  outputs_traffic_light/iter0/response1/traffic_light_fsm.sv \
  traffic_light_tb.v
yanbowang@YanbodeMacBook-Air traffic_light_example % vvp -n outputs_traffic_light/iter0/response1/traffic_light_fsm.vvp
Test case 1 passed - FSM transitioned to    RED after 0 clock cycles.
Test case 2 passed - FSM transitioned to  GREEN after 32 clock cycles.
Test case 3 passed - FSM transitioned to YELLOW after 20 clock cycles.
Test case 4 passed - FSM transitioned to    RED after 7 clock cycles.
All test cases passed!
Mismatches: 0 in 4 samples
```

Figure 50: Icarus Verilog compilation and simulation output for iteration 0, response 1

2 8-Bit CPU

For the 8-bit CPU prompt, in addition to the initial prompt given in the PowerPoint, I obtained the complete ISA and some crucial microarchitecture details from the following GitHub repository: `https://github.com/kiwih/tt03-verilog-qtcoreA1`.

I wrote a testbench covering the instructions in the ISA, although HLT was executed without an explicit check. I included each instruction's function and opcode in the prompt so that the model had the information needed to implement them.

Here is the entire prompt:

```
//Let us make a brand new microprocessor design together. We're severely constrained on space and I/O.
//We have to fit in 1000 standard cells of an ASIC,
//so I think we will need to restrict ourselves to an accumulator based 8-bit architecture with no multi-byte instructions.
//
//The microarchitecture is suggested complete an instruction in 2 stages
//The instruction shall implement the following ISA:
//
//Memory instructions use a 3-bit opcode and a 5-bit address:
//000: LDA - load M[address] into ACC.
//001: STA - store ACC in M[address].
//010: ADD - add M[address] to ACC.
//011: SUB - subtract M[address] from ACC.
//100: AND - bitwise AND.
//101: OR - bitwise OR.
//110: XOR - bitwise XOR.
//
//1110iiii: ADDI - add the unsigned 4-bit immediate iiii to ACC.
//
//The following are complete instruction bytes:
//F0: JMP - jump to ACC.
//F1: JSR - save the next instruction address in ACC and jump to the old ACC.
//F2: BEQ_FWD - if ACC is zero, add 2 to the already-incremented PC.
//F3: BEQ_BWD - if ACC is zero, subtract 3 from that PC.
//F4: BNE_FWD - same forward branch when ACC is nonzero.
//F5: BNE_BWD - same backward branch when ACC is nonzero.
//F6: SHL - shift ACC left by 1.
//F7: SHR - shift ACC right by 1.
//F8: SHL4 - shift ACC left by 4.
//F9: ROL - rotate ACC left by 1.
//FA: ROR - rotate ACC right by 1.
//FB: LDAR - load M[ACC] into ACC.
//FC: DEC - subtract 1 from ACC.
//FD: CLR - clear ACC.
//FE: INV - invert every ACC bit.
//FF: HLT - stop execution until reset.
//
//Use an 8-bit register named acc.
//Use a 5-bit register named pc.
//Use a shared instruction/data memory named mem with 32 entries of 8 bits.



module cpu_8_bit (
    input clk,
    input reset_n
);
```

Figure 51: CPU prompt, ISA requirements, and module template

The parameter setup is included in the config.json file:

```
{
    "general": {
        "prompt": "./cpu_8_bit.sv",
        "name": "cpu_8_bit",
        "testbench": "./cpu_8_bit_tb.v",
        "model_family": "Portkey",
        "model_id": "@vertexai/anthropic.claude-sonnet-4-5@20250929",
        "num_candidates": 5,
        "iterations": 5,
        "outdir": "outputs_cpu_8_bit",
        "log": "log.txt",
        "ensemble": false
    }
}
```


Figure 52: AutoChip configuration for the 8-bit CPU

The model I used for this example is Claude Sonnet 4.5, with an iteration limit of 5 and 5 candidates per iteration. Here is the printed result from the code cell in the notebook.

```
Iteration: 0
Model type: Portkey
Model ID: @vertexai/anthropic.claude-sonnet-4-5@20250929
Number of responses: 5
Testbench ran successfully
Mismatches: 0
Samples: 24
Cost for response 0: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 24
Cost for response 1: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 24
Cost for response 2: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 24
Cost for response 3: $0.0000000000
Testbench ran successfully
Mismatches: 0
Samples: 24
Cost for response 4: $0.0000000000
Response ranks: [1.0, 1.0, 1.0, 1.0, 1.0]
Response lengths: [3237, 3517, 3308, 3407, 3491]
```

Figure 53: Iteration 0 results: all five CPU candidates passed all 24 checks

All five candidates passed the testbench with no mismatches in the first iteration. AutoChip selected response number 0 as the best response because it had the shortest response length. No correction round was needed.

The code uses two stages, as specified in the prompt. It first fetches the instruction and increments the program counter, then executes the operation to update the accumulator, memory, or program counter. Reset clears the accumulator and program counter before each test, while the testbench loads instructions and data directly into memory. The generated design produced the expected results for all 24 checks.

Here are the iverilog and vvp commands used, including the simulation output:

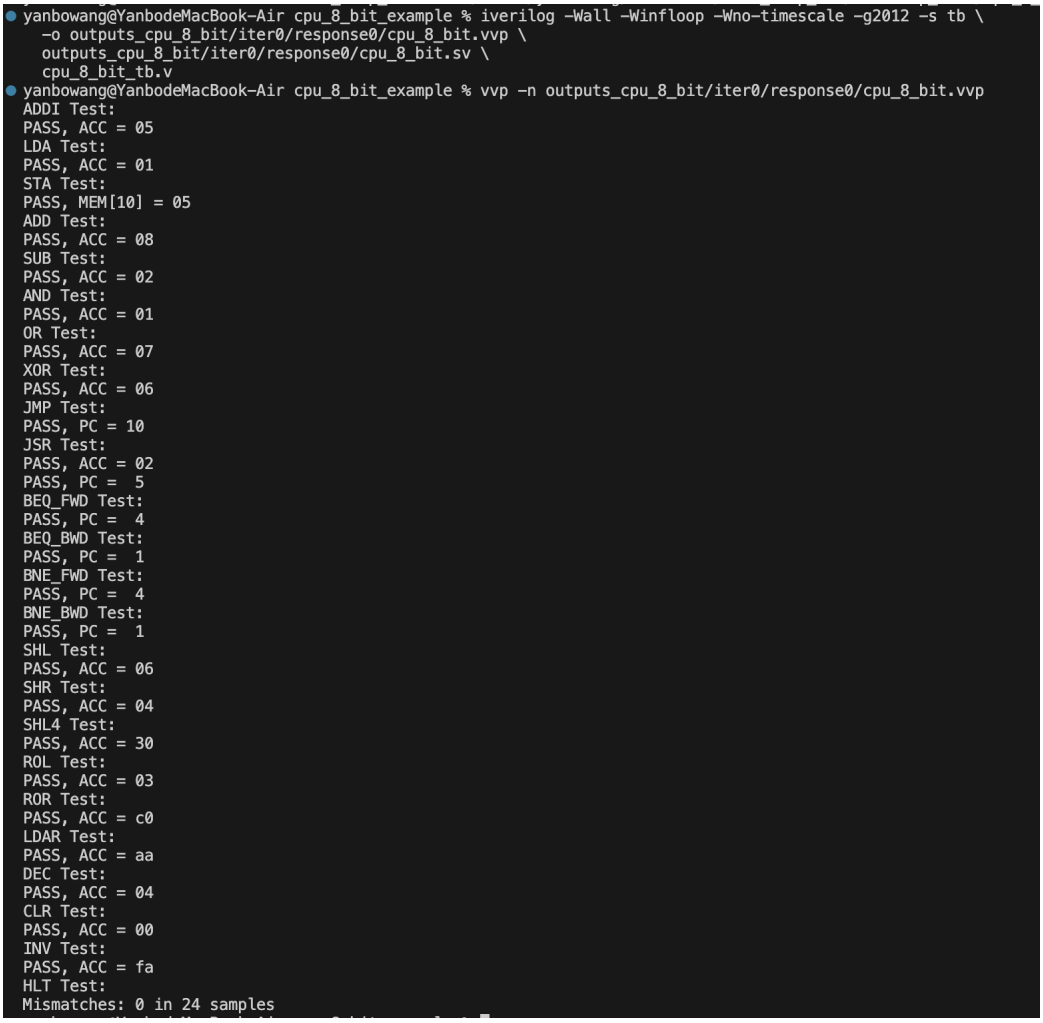


Figure 54: Icarus Verilog compilation and simulation output for iteration 0, response 0